



Технически Университет - София

Тема: Uber приложение (C# + MongoDB)

курсова работа по
“Програмиране на C#”

Имена на студент: **Лъчезар Тодоров Димитров**

Факултетен номер: **121222037**

Група: **39 б**

Специалност: **Компютърно и софтуерно инженерство**

Образователно-квалификационна степен: **бакалавър**

1. Въведение

Настоящият проект представя разработката на база данни за ride-sharing приложение, подобно на Uber. Основната идея на системата е да свързва потребители, които се нуждаят от транспорт, с шофьори, които предоставят такава услуга.

Приложението позволява на потребителите да заявяват курсове, които след това се поемат от налични шофьори. Всеки курс има определен статус (например заявен, изпълняван, завършен или отменен), както и информация за начална и крайна точка, цена и време. Освен това системата поддържа информация за плащанията, свързани с всеки курс.

1. Бизнес проблем

В съвременните градове придвижването на хора е свързано с редица предизвикателства като натоварен трафик, ограничена наличност на транспорт и неефективно използване на наличните ресурси. Традиционните таксиметрови услуги често не успяват да отговорят на нарастващото търсене

2. Цели на проекта

Основната цел на проекта е създаване на работещ модел на база данни за ride-sharing система, използвайки MongoDB и C# (.NET).

Конкретните цели включват:

- Проектиране на подходяща структура на база данни
- Създаване на основни колекции – потребители, шофьори, курсове и плащания
- Демонстриране на CRUD операции (създаване, четене, актуализиране и изтриване)
- Използване на заявки за филтриране и търсене на данни чрез MongoDB Driver в C#
- Реализация на агрегиращи заявки за анализ на информация (например приходи по шофьор)
- Демонстриране на контрол на достъпа чрез ролево-базиран модел (RBAC)

3. Защо нерелационна база данни

Изборът на нерелационна база данни (MongoDB) е обусловен от необходимостта от гъвкавост и висока производителност.

Ride-sharing приложенията изискват висока производителност и обработка на голям брой заявки в реално време. MongoDB е оптимизирана за бързо четене и запис, което я прави подходяща за такива сценарии в сравнение с релационни системи, при които JOIN операциите могат да бъдат по-бавни. В процеса на развитие на приложението могат да се добавят нови полета (например рейтинг, история на курсове, допълнителни атрибути), без необходимост от миграции на база данни и промяна на цялата схема.

Допълнително предимство на MongoDB е вградената поддръжка на географски данни, което е ключово за ride-sharing приложения, където се използват GPS координати за намиране на най-близък шофьор.

4. Използвани технологии

- MongoDB – основна база данни
- MongoDB Driver for C# (.NET) – за комуникация с базата данни
- MongoDB Shell (mongosh) – за изпълнение на заявки
- MongoDB Compass – визуален инструмент за работа с базата
- C# (.NET Console Application) – реализация на бизнес логика

2. Архитектура и реализация

4. Структура на базата данни

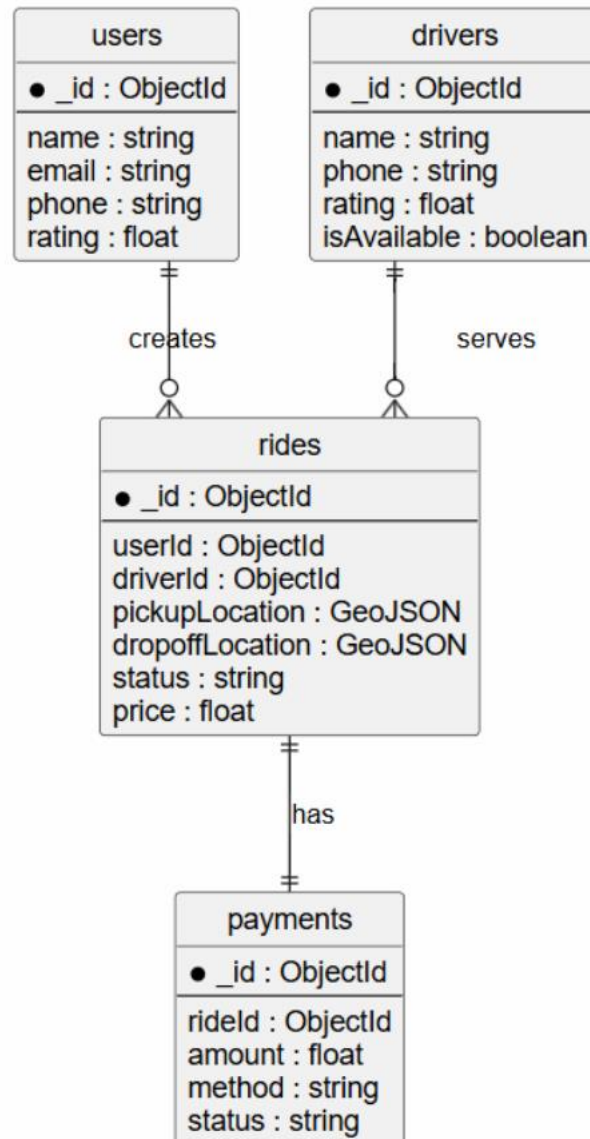
Базата данни съдържа четири основни колекции:

users – съдържа информация за потребителите
drivers – съдържа информация за шофьорите и техните автомобили

rides – съдържа информация за курсовете
payments – съдържа информация за плащанията

Връзките между колекциите се реализират чрез ObjectId:

- Един потребител може да има много курсове
- Един шофьор може да изпълнява много курсове
- Всеки курс има едно плащане



users → rides (1:N)

drivers → rides (1:N)

rides → payments (1:1)

3. CRUD операции

В проекта са реализирани основните операции за работа с данни:

3.1 Създаване (Create) – добавяне на нови курсове

```
var ride = new Ride
{
    UserId = user.Id,
    DriverId = driver.Id,
    Price = new Random().Next(5, 50),
    Status = "completed",
    RequestedAt = DateTime.Now.AddMinutes(-20),
    CompletedAt = DateTime.Now
};

rides.InsertOne(ride);

Console.WriteLine($"Ride created: {user.Name} -> {driver.Name}");
```

3.2 Четене (Read)

```
static void ShowRides(IMongoCollection<Ride> collection)
{
    var rides = collection.Find(_ => true).ToList();

    foreach (var r in rides)
    {
        Console.WriteLine($"ID: {r.Id} | Price: {r.Price} | Status: {r.Status}");
    }
}
```

3.3 Актуализиране (Update)

```
static void UpdateRide(IMongoCollection<Ride> collection)
{
    Console.Write("Enter Ride ID: ");
    var id = Console.ReadLine();

    var filter = Builders<Ride>.Filter.Eq("_id", ObjectId.Parse(id));
    var update = Builders<Ride>.Update.Set("Status", "completed");

    collection.UpdateOne(filter, update);

    Console.WriteLine("Updated!");
}
```

3.4 Изтриване (Delete)

```
static void DeleteRide(IMongoCollection<Ride> collection)
{
    Console.WriteLine("Enter Ride ID: ");
    var id = Console.ReadLine();

    var filter = Builders<Ride>.Filter.Eq("_id", ObjectId.Parse(id));
    collection.DeleteOne(filter);

    Console.WriteLine("Deleted!");
}
```

4. Агрегиращи заявки

MongoDB Aggregation Pipeline чрез MongoDB C# Driver

Изчисляване на общите приходи от курсове

```
static void RevenuePerDriver(IMongoCollection<Ride> collection)
{
    var result = collection.Aggregate()
        .Match(r => r.Status == "completed")
        .Group(r => r.DriverId, g => new
        {
            DriverId = g.Key,
            Total = g.Sum(x => x.Price)
        })
        .SortByDescending(x => x.Total)
        .ToList();

    foreach (var r in result)
    {
        Console.WriteLine($"Driver: {r.DriverId} | Revenue: {r.Total}");
    }
}
```

Изчисляване на Брой завършени пътувания по шофьор

```
static void CompletedRidesPerDriver(IMongoCollection<Ride> collection)
{
    var result = collection.Aggregate()
        .Match(r => r.Status == "completed")
        .Group(r => r.DriverId, g => new
        {
            DriverId = g.Key,
            TotalRides = g.Count()
        })
        .SortByDescending(x => x.TotalRides)
        .ToList();

    Console.WriteLine("COMPLETED RIDES PER DRIVER");

    foreach (var r in result)
    {
        Console.WriteLine($"Driver: {r.DriverId} | Rides: {r.TotalRides}");
    }
}
```